

LARS JANSEN

Senior Software Engineer

Assen, Drenthe, Netherlands | +31 97058049397 | larsjansen0521@outlook.com | linkedin.com/in/lars-jansen-0a37b8393

PROFESSIONAL SUMMARY

Senior Full-Stack Engineer with 9+ years building **React** (v16.8–v18) and **Node.js** (v14–v20) platforms that ship reliable customer workflows in SaaS, fintech-style billing, and operations dashboards. I design **TypeScript** UI architecture, secure **REST** APIs, and event-driven services with **PostgreSQL** (v11–v15), **Redis** (v5–v7), and **Kafka** (v2.x), then harden delivery with **Docker**, **Kubernetes**, and CI/CD. Known for untangling legacy JS → modern hooks, migrating monolith endpoints into modular services, fixing memory leaks, race conditions, and slow queries, and turning messy incidents into measurable **SLOs** with tracing and actionable alerts.

SKILLS

- Frontend: **React** (v16.8–v18), **TypeScript** (v4.x–v5.x), JavaScript (ES2017+), SPA performance, WebSockets, Vite/webpack, testing with **Jest** (v27–v29) + React Testing Library (v12–v14)
- Backend & APIs: **Node.js** (v14–v20), Express (v4.x), NestJS (v7–v10), **REST**, OAuth2/OIDC, **JWT**, RBAC, rate limiting, background workers, API contract testing (Pact-style)
- Data / Messaging / Search: **PostgreSQL** (v11–v15), MySQL (v5.7–v8.x), Redis (v5–v7), DynamoDB, **Kafka** (v2.x), RabbitMQ (v3.x), AWS SQS/SNS, Elasticsearch (v7.x), OpenSearch (v1.x–v2.x)
- Cloud & DevOps: **AWS** (EKS/ECS/EC2/Lambda/API Gateway/S3/RDS/CloudFront/ IAM/EventBridge/Step Functions), Docker (v19–v25), Kubernetes (v1.20–v1.28), Terraform (v1.x), Helm (v3.x), GitHub Actions / Azure DevOps CI/CD
- Observability / Quality / Security: **OpenTelemetry** (v1.x), Datadog, Prometheus (v2.x), Grafana (v8–v10), ELK/EFK, CloudWatch, audit logging, secrets hygiene, incident runbooks

WORK HISTORY

Senior Software Engineer RavenTrack

August 2021 to November 2025

- I led a **React** (v17–v18) rebuild of a multi-tenant admin console, replacing brittle class components with hooks to cut UI regressions and simplify releases.
- I built **Node.js** (v16–v20) **REST** services in **TypeScript**, standardizing controllers, validation, and error contracts so frontend and backend shipped in lockstep.
- I migrated legacy endpoints into modular services behind **API Gateway**, using feature flags and shadow traffic to avoid downtime while moving critical flows.
- I fixed a production memory leak caused by unbounded in-memory caching, then added heap snapshots and guardrails to prevent repeats during peak load.
- I resolved intermittent “double charge” bugs by enforcing idempotency keys, tightening DB constraints, and adding retry-safe workflows across payment events.
- I optimized slow dashboards by redesigning **PostgreSQL** indexes, rewriting N+1 queries, and introducing read replicas for heavy reporting workloads.
- I introduced event-driven processing with **Kafka** (v2.x) and AWS **SQS/SNS**, separating user requests from heavy jobs and improving tail latency.
- I improved authentication by aligning **OAuth2/OIDC** flows with short-lived **JWT** tokens, adding refresh logic, and tightening **RBAC** around sensitive actions.
- I reduced incident noise by rolling out **OpenTelemetry** tracing and structured logs, then wiring alerts to actionable SLO-based thresholds.
- I implemented rate limiting and abuse detection at the edge, preventing bot spikes from degrading service while keeping legitimate traffic unaffected.
- I hardened deployments with **Docker** and **Kubernetes** (EKS), adding health probes, resource limits, and safe rollout strategies for noisy neighbors.
- I built CI/CD gates in GitHub Actions with unit, integration, and contract tests, stopping breaking API changes before they reached production.
- I handled a brittle migration from older Node runtimes to newer LTS by fixing native module builds, tightening dependency locks, and updating pipelines.
- I shipped new product features end-to-end (React UI + API + data model) by pairing with PM/design and turning ambiguous specs into shipped outcomes.
- I mentored engineers through code reviews, debugging playbooks, and performance profiling, raising team quality without slowing delivery speed.
- I delivered customer-facing features using **React** (v16.8–v17) and **Node.js** (v12–v16), focusing on clean component boundaries and predictable state.

Software Engineer Infinite Loop

June 2018 to July 2021

- I migrated a legacy jQuery-heavy UI into a modern SPA, reducing duplicated logic and introducing reusable UI patterns to speed new screens.

- I fixed hard-to-reproduce UI race conditions by stabilizing async flows, adding deterministic tests, and improving client-side error handling.
- I built API integrations and backend jobs with Express (v4.x), ensuring retries, timeouts, and consistent error codes across downstream systems.
- I stabilized file uploads by streaming to S3, validating MIME types, and adding antivirus hooks so large payloads no longer crashed servers.
- I improved search and filtering using **Elasticsearch** (v6–v7), tuning analyzers and pagination to avoid deep-scan performance cliffs.
- I reduced query hotspots by redesigning schemas and indexes in **PostgreSQL**, then measuring impact with explain plans and benchmarks.
- I introduced caching with **Redis** (v5), carefully handling cache invalidation to fix stale data bugs that confused support teams.
- I upgraded build and test tooling to strengthen quality, adding **Jest** suites and smoke tests that caught breaking changes early.
- I supported on-call by writing runbooks and adding dashboards, turning recurring incidents into measurable fixes instead of repeated firefights.
- I collaborated with QA and product to ship incremental releases, using feature toggles to deliver value without risky “big bang” launches.

Software Developer Datamart

October 2015 to May 2018

- I built internal web tools using server-rendered JS and early **React** patterns, gradually introducing components where it reduced duplication safely.
- I created backend APIs in **Node.js** (v8–v12) and integrated them with reporting workflows, keeping responses consistent for legacy consumers.
- I fixed recurring “missing data” defects by adding stronger validations, backfills, and reconciliations between ETL outputs and UI expectations.
- I migrated a fragile cron-based pipeline into a queued worker model, improving reliability and making failures visible instead of silent.
- I improved SQL performance by adding indexes and rewriting reports, reducing timeouts that blocked finance and operations from daily work.
- I implemented auditing and permission checks, preventing unauthorized exports and giving admins clear traceability for sensitive changes.
- I introduced basic monitoring and log correlation so production issues could be diagnosed quickly without guesswork or manual DB poking.
- I handled an incremental migration from older libraries to newer stable releases by locking dependencies, testing critical paths, and rolling out safely.
- I partnered with stakeholders to translate messy business rules into maintainable code, documenting edge cases so future changes stayed predictable.

EDUCATION

Bachelor's degree in Computer Science University of Amsterdam

2011 to 2015

- Built strong foundations in software engineering, data structures, and systems design, with coursework that supports designing reliable services and data-driven applications.

PROJECTS

Real-Time Ops Console (React + Node)

- Built a near real-time monitoring console with **React** (v18) and WebSockets, surfacing incident signals and reducing mean time to detect.
- Implemented a Node event pipeline with **Kafka** (v2.x) plus **PostgreSQL** (v14–v15), enabling replayable processing and reliable audit history.